Database & Login Troubleshooting Runbook

WhoIsDésir® Media Agency Platform — Developer Training

Purpose. This runbook walks through the exact steps to diagnose and fix the most common failure mode in this platform: the site loads fine but nobody can log in. Login failures here are almost always a **database connection problem**, not an app or hosting problem.

1. Symptom

The deployed site loads and renders the login form, but every attempt to sign in shows one of:

- **"Invalid credentials"** — even when the email and password are known to be correct.
- **"Connection error. Please try again."**

2. Why the error message lies

The credentials provider runs a database lookup on every sign-in attempt (`src/lib/auth.ts` , inside `authorize()`):

```
const user = await prisma.user.findUnique({ where: { email: credentials.email } });
if (!user) return null;
const valid = await bcrypt.compare(credentials.password, user.passwordHash);
if (!valid) return null;
```

If the database is unreachable, the `findUnique()` call throws. The login page (`src/app/login/page.tsx`) converts *any* failure into "Invalid credentials" — so a dead database looks identical to a wrong password. **Always verify the database before resetting a password.**

3. Confirm it is data, not hosting

1. Fetch the login page, e.g. `https://whoisdesirmediaagency.vercel.app/login` .

2 If the form renders, the Next.js app and Vercel deployment are healthy. The problem is the **database connection** — continue below.

---

4. Inspect the environment variables

Two variables control the database connection. Read them (never print their values into logs or chat):

Variable	Host	Port	Used by
`DATABASE_URL`	`...pooler.supabase.com`	`6543` (+ `?pgbouncer=true`)	The running application
`DIRECT_URL`	`db.<project-ref>.supabase.co`	`5432`	`prisma db push` / migrations

Common mistake: pointing `DIRECT_URL` at the *pooler* host on port 5432. Prisma migrate/push will fail or hang. The direct connection must use the `db.<project-ref>.supabase.co` host.

5. DNS test the project reference

Each Supabase project has a unique reference id embedded in the URL, e.g. `postgres.<project-ref>`. Test whether the direct host resolves:

```
host <project-ref>.db.supabase.co
```

- **NXDOMAIN** → the Supabase project has been deleted or renamed. A new project will have a **different ref**.
- Resolves to an IP → the project exists, continue to the pooler test.

6. Pooler test

The pooler host resolves even when the project is gone (it is shared infrastructure). The real signal is the server response:

```
FATAL: tenant/user postgres.<project-ref> not found
```

> This exact error means the pooler is reachable but **no project is registered under that ref** — the project was deleted or recreated. This is the failure we observed for ref `zwepuifkjvffjtiembia`.

7. Get the correct connection string

- **Supabase** → project → *Settings* → *Database* → *Connection string*. The **URI** tab has both a "Connection pooling" section (your `DATABASE_URL`) and a "Direct connection" section (your `DIRECT_URL`).
- **Vercel** → project → *Settings* → *Environment Variables* — fastest place to copy the values already in use.
- If you only have one string, derive the other: keep the same user, password, and database name; change only the host and port as shown in section 4.

8. Apply the fix

1. Update both `DATABASE_URL` and `DIRECT_URL` in `.env`.
2. Delete any duplicate `DATABASE_URL` lines — dotenv lets the **last** definition win, so a stale duplicate silently overrides the good one.
3. If your local `.env` differs from Vercel's environment, sync both — production login uses the Vercel values.

> **Never commit `.env`.** It is gitignored (`.gitignore` contains `.env*`). Never paste credentials into chat, tickets, or logs. Treat any exposed database password as compromised and **rotate it in Supabase** (Settings → Database → Reset password).

9. Verify the connection

Before touching any data, confirm Prisma can connect, then count users:

```
node -e "const{PrismaClient}=require('@prisma/client');const p=new
PrismaClient();p.user.count().then(n=>{console.log('users:',n);return
p.\$disconnect();}).catch(e=>{console.log('FAIL',e.message);process.exit(1);})"
```

A number printed is success. A `FATAL` or timeout means the URL is still wrong.

10. Rebuild data if the database was recreated

A brand-new Supabase project starts empty. Reapply the schema and seed:

```
npm run db:push    # apply schema (no migration files in this project)
npm run db:seed    # creates baseline users from prisma/seed.ts
```

Seed logins: `admin@whodesir.com` / `password` and `staff@whodesir.com` / `password`. Note the spelling — `whodesir` (not `whoisdesir`) — and always change these after a production seed.

11. Create or reset an admin account

If an admin login is missing or its password is unknown, use the helper script:

```
npx tsx prisma/create-admin.ts
```

It upserts `admin@whodesir.com`, generates a strong 16-character password, and prints it **once**. Copy it immediately and store it in your password manager, then rotate it if it was ever displayed anywhere shared.

12. Safety rules

- Never print, log, or commit database credentials or passwords.
- Rotate any password that has been exposed, immediately.
- Production environment variables live in **Vercel**; local `.env` is for development only.
- Change seeded passwords before any real deployment.

13. Quick reference

Symptom	Likely cause	Fix
"Invalid credentials" for a known-good password	Dead / unreachable database	Check `DATABASE_URL`, DNS, pooler response
`tenant/user ... not found`	Project ref deleted or recreated	Get the new ref's connection string from Supabase
`NXDOMAIN` on `db.<ref>.supabase.co`	Supabase project gone	Create/restore project, update both URLs
`prisma db push` hangs or fails	`DIRECT_URL` points at pooler host	Use `db.<ref>.supabase.co:5432`
Admin account missing after DB reset	No seed data	`npx tsx prisma/create-admin.ts`